POSTASSIST LK · J26-IT-372 · INDIVIDUAL COMPONENT

Technical Reference

Every folder, file, technology, model and algorithm of the Smart Registered Mail Intake and Tracking component – what it is, what it does and how it works, in simple words.

WHAT IS INSIDE

1 The folder map

2 Technologies and why each was chosen

3 The trained models, layer by layer

4 The external engines (Tesseract, optional LLM)

5 Algorithms – images, OCR and addresses

6 Algorithms – weighing and price

7 Algorithms – scans, GPS and messages

8 Algorithms – security and identifiers

9 How the models were evaluated

10–16 Every file, folder by folder

HOW TO USE THIS REFERENCE

Each item says **what it is**, **what it is for** and **where it lives**. Formulas are shown once, in plain notation, with an example where it helps. Numbers (versions, thresholds, layer sizes) are copied from the code and model files as they are on 20 October 2026. For the overall story read the Component Guide; for the device read the IoT guide.

Component owner

Senarathna W.L.S.K. · IT23324138

B.Sc. (Hons) in Information Technology, SLIIT

Edition

English · 20 October 2026

PART 1

The folder map

The repository, top level first. Folders that belong to teammates are marked.

```
J26-IT-372/
├── backend/                  the server (Python, FastAPI) – one process for the whole system
│   ├── app/                  MY COMPONENT: the registered-mail API
│   │   ├── main.py, config.py, db.py, security.py, audit.py,
│   │   │   schemas.py, seed.py, timeutil.py     start-up and shared code
│   │   ├── reference/        gazetteer, post offices, vocabulary, 2026 tariff
│   │   ├── routers/          the API "doors" (11 files, 70 endpoints)
│   │   └── services/         the rules and algorithms (tariff, weighing, scans, OCR …)
│   ├── models/               trained models used at run time (CRF, script-ID, HTR)
│   ├── tests/                71 automated tests of my component
│   ├── tools/                start scripts, IoT simulator, IoT bridge
│   ├── storage/              photos and e-mail files written at run time (not in git)
│   ├── main.py, routes.py, db.py, attendance.py, payroll.py,
│   │   productivity.py, excel_io.py   TEAMMATE: Postal KPI desk API
│   ├── requirements.txt      exact Python package versions
│   └── .env, .env.example    settings and secrets (.env is not in git)
├── frontend/                 the web app (React + Vite)
│   └── src/
│       ├── App.jsx · main.jsx · api.js · auth.jsx · index.css
│       ├── pages/            MY registered-mail pages (15)
│       ├── components/       MY shared parts (layout, address, image, weight …)
│       └── admin/, attendance/, entitlements/, components/ui/
│                             TEAMMATE: KPI desk pages
├── mobile_app/
│   ├── App.js · src/         Expo React Native field app (postman / hub scans, offline)
│   └── flutter_app/          Flutter staff app (all roles, Sinhala / Tamil / English)
├── iot/postassist_scale_cam/ ESP32-CAM firmware (scale + camera)
├── ml/                       dataset builders, model training, evaluation, reports
├── Datasets/                 raw downloads, curated files, processed data, collection templates
└── Documents/                requirements, guides, slides, PDFs
```

OUTSIDE THE REPOSITORY

`%USERPROFILE%\PostAssist\` holds the portable MongoDB (`mongodb\bin\mongod.exe`), the database files (`mongo-data`), the handwriting datasets and training cache (`datasets\`) – kept out of the OneDrive-synced repository because they are large.

PART 2

Technologies and why each was chosen

Versions as installed and tested.

2.1 Backend

Technology	What it is · what it does here
Python 3.14	Programming language of the backend and of all model training – one language for API and AI.
FastAPI 0.142	Web framework: turns Python functions into API endpoints, checks every request against a schema and generates the `/docs` page automatically.
Pydantic (comes with FastAPI)	Defines the exact shape of each request (`schemas.py`): types, minimum lengths, ranges. Bad input is refused with code 422 before any code runs.
Uvicorn 0.54	The web server that runs the FastAPI app on port 8000.
MongoDB 8.2 + **PyMongo 4.18**	Document database and its Python driver. A letter is one document with its events and messages inside, so each change is one atomic write. Unique indexes enforce "no double records"; a TTL index deletes old scale readings.
PyJWT 2.15	Creates and checks signed login tokens (JWT, HS256).
python-multipart	Receives uploaded photo files.
NumPy 2.5 · **OpenCV 5.0**	Number arrays and image processing: decoding photos, quality metrics, thresholding, straightening, line finding.
ONNX Runtime 1.30	Runs the trained neural networks (script-ID and handwriting) fast on the CPU, without PyTorch.
python-crfsuite 0.9.12	Runs (and trains) the CRF address parser.
pytesseract 0.3 + **Tesseract 5.5**	The printed-text OCR engine with the `sin` , `tam` and `eng` language packs.
python-barcode 0.16 · **qrcode 8.2**	Draw the Code 128 barcode and the QR codes as SVG.
pytest 9.1 · **httpx**	Automated tests; httpx lets the tests call the API like a client.
Standard library: `hashlib` , `hmac` , `secrets` , `decimal` , `threading` , `smtplib`	Password hashing, constant-time comparison, secure random IDs, exact money arithmetic, the outbox worker thread, SMTP e-mail.

2.2 Web app

Technology	What it is · what it does here
React 19	JavaScript library for building the screens out of reusable components that update themselves when data changes.

Technology	What it is · what it does here
React Router 7	Maps addresses (`/counter` , `/items/:id` , `/track/:id` …) to pages; protects pages behind login.
Vite 8 (+ `plugin-basic-ssl`)	Development server and bundler; serves the app on `https://localhost:5173` and forwards `/api` to the backend. HTTPS is needed for the webcam.
Tailwind CSS 4	Styling with small utility classes; the navy theme colours are defined once in `index.css` .
ESLint 9	Checks the code for mistakes.
MUI, shadcn, lucide-react, qrcode.react	Used by the **teammate's** KPI desk pages, not by the registered-mail pages.

2.3 Phone apps

Technology	What it is · what it does here
Expo SDK 57 · **React Native 0.86**	Build an Android/iOS app with React; Expo Go runs it on a phone without an app store.
expo-camera · expo-location · expo-sqlite · expo-secure-store · expo-crypto · NetInfo · react-native-svg	Barcode/QR scanning · GPS with accuracy and mock flag · the offline scan queue · storing the login token safely · random event IDs · noticing when the network returns · the signature drawing.
Flutter 3.47 · **Dart 3.13**	Google's app toolkit; the staff app in `mobile_app/flutter_app/` .
provider · http · sqflite · mobile_scanner 7 · geolocator 14 · barcode_widget · connectivity_plus · flutter_secure_storage · image_picker · shared_preferences · uuid · intl / flutter_localizations	App state · API calls · offline queue · barcode/QR scanning · GPS · drawing labels · network changes · token storage · camera/gallery photos · settings · event IDs · Sinhala/Tamil/English texts and date formats.

2.4 IoT device

Technology	What it is · what it does here
Arduino C++ · ESP32 Arduino core 3.3	Language and system libraries for the ESP32-CAM.
ArduinoJson 7	Builds the JSON of each reading and reads the server's reply.
WiFi · HTTPClient · WiFiClientSecure · Preferences · WebServer · esp_camera	Wi-Fi and set-up network · requests to the backend · HTTPS · saving calibration in flash · the device's own page · the OV3660 camera.
Arduino IDE 2 / arduino-cli	Compiling and uploading the firmware.

2.5 Machine learning

Technology	What it is · what it does here
PyTorch 2.11 + CUDA 12.8	Trains the neural networks on the laptop's RTX 2050 (4 GB) GPU.
ONNX	Standard file format the trained networks are exported to.
uharfbuzz + **freetype-py**	Correct shaping and drawing of Sinhala and Tamil text when creating training images (Pillow on Windows cannot shape conjuncts).
Pillow · scikit-learn · pyarrow · matplotlib	Image files · metrics and splits · reading the IAM parquet files · charts in the reports.

PART 3

The trained models, layer by layer

Three models I trained. All are in `backend/models/`, each with a `.meta.json` that records how it was made.

3.1 Script identification – a CNN

Job	Look at one text line and say **Sinhala, Tamil or English**, so Tesseract can use the right language pack.
Input	A grey line image resized to **32 × 256 pixels** (keeping the shape, padded with paper colour), values scaled to −1 … 1.
Layers	Conv 3×3 (32) → pool → Conv (64) → pool → Conv (128) → Conv (128) → pool → Conv (192) → **global average pooling** → dropout 0.3 → linear → 3 scores. Every Conv is followed by batch normalisation and ReLU. **462,755** parameters.
Training	45,000 rendered lines (18 fonts), AdamW, learning rate 2×10^{-3} with a one-cycle schedule, batch 128, 14 epochs, 3 random seeds; the seed with the best *validation* accuracy is shipped.
Calibration	**Temperature scaling**: one number T (0.547) is fitted on validation so the probabilities match reality (see 5.12).
In use	`ocr/script_id.py`: every line of the block is classified; a width-weighted vote gives the block's script; English is always added for local-script blocks (house numbers, "No.").
Result	98.6 % on 6 fonts never seen; 2–7 ms per line on a CPU.

IN SIMPLE WORDS – WHAT A CNN DOES

A convolutional neural network slides small 3 × 3 "pattern detectors" over the image. Early layers find strokes and curves, deeper layers combine them into shapes typical of each script (Sinhala loops, Tamil angles, Latin straight strokes). Pooling shrinks the image so later layers see more at once. At the end, global average pooling summarises the whole line into 192 numbers and a final layer turns them into three scores.

3.2 Address-field parser – a linear-chain CRF

Job	Give every word of an address one label: NAME, HOUSE, STREET, LOCALITY, CITY, REGION, POSTCODE, COUNTRY or O (other).
Input	The words (tokens) of the address with their line number, and about 40 **clues (features)** per word (5.10).
Model	A **Conditional Random Field**: it learns a weight for every (clue, label) pair and for every (label → next label) pair, and picks the label sequence with the highest total score for the whole address at once.
Training	24,245 addresses; L-BFGS optimiser with L1 (c1) and L2 (c2) penalties. Five (c1, c2) pairs were tried on an 8,000-address subsample and judged on validation; chosen c1 = 0.1, c2 = 0.01.
Confidence	The **marginal probability** of each word's label; a field's confidence = its lowest word probability. Review threshold 0.974, fixed on validation.
In use	`address/parser.py`; falls back to simple rules if the model file is missing.
Result	99.4 % required-field exact match on towns never seen (rules: 57.5 %); ECE 0.0024.

> **IN SIMPLE WORDS – WHY A CRF**
>
> The label of a word depends on its neighbours: "25" after "No." is a house number, "03" after "Colombo" is part of the city. A CRF scores whole sequences, so it learns such patterns (e.g. NAME is followed by HOUSE, POSTCODE comes last). It needs little data, trains in minutes on a CPU, its weights can be inspected, and its probabilities are well calibrated.

3.3 Handwriting recogniser – a CRNN trained with CTC

Job	Read one handwritten line (Sinhala, English, Tamil or mixed) into text.
Input	The cleaned ink image of the line (5.5), **48 pixels high** and as wide as the writing (up to 1,280 px).
Layers	**7 convolution layers** (48, 96, 160, 160, 256, 256 channels with batch norm + ReLU and pooling, then a 3 × 1 layer to 320) turn the image into a sequence of feature columns, one per 4 pixels of width → dropout 0.2 → **2-layer bidirectional LSTM** (256 per direction, dropout 0.25) reads that sequence forwards and backwards → linear layer → scores for **195 characters + 1 "blank"** for every column. **4,478,612** parameters.
Loss	**CTC** (Connectionist Temporal Classification): lets the network learn from line images and their text without anyone marking where each letter is (5.8).
Training	Real SinOCR (820 lines) and IAM (6,482 lines) + freshly drawn synthetic lines every epoch; AdamW, learning rate 1.5×10^{-3} with warm-up then cosine decay, weight decay 10^{-4}, mixed precision, batch 32, 40 epochs (best: epoch 37). **Augmentation**: slant/shear, slope, width scaling, thicker or thinner pen, blur, darkness (gamma), ruled or dotted form lines, noise, dirt specks.
Selection	Lowest average character error rate on the three validation sets (real Sinhala, real English, Tamil in a validation font). Test sets read only once, after training.
In use	`ocr/htr.py`: greedy CTC decoding with a digit preference inside numbers, Sinhala joiner restoration, per-word confidence. 13.1 ms per line on CPU.

Result	CER 18.1 % (real Sinhala), 9.2 % (real English); Tesseract 91 % / 55 %.

IN SIMPLE WORDS – CNN + LSTM + CTC

The CNN part "looks" at the writing; the LSTM part "reads" the columns in order, remembering what came before and after (so it can tell that a curve belongs to "ක" and not "ත"); CTC turns the column-by-column guesses into text by merging repeats and dropping blanks. Because one network learned all three scripts, a mixed line like "No. 230, සිංහපුර" needs no switching.

PART 4

The external engines

Used, not trained by me.

Engine	What it is and how it is used
Tesseract 5.5	Open-source OCR for printed text. Its modern engine is also an LSTM network. Called with `--oem 1` (LSTM engine) and `--psm 6` (one uniform block of text); returns words with confidence 0–100 and boxes. In B1 all three language packs are used together; in P1 only the packs the script model chose.
Gemma 3 4B IT via Ollama (optional, off)	A small instruction-tuned language model run locally. Given the address and three worked examples (English, Sinhala, Tamil), it returns the fields as JSON (temperature 0). Its output must pass the **guard** (5.13). Off by default (`LLM_PROVIDER=none`).
OpenStreetMap data (offline copy)	Post-office points and street names, used as reference data; no map service is called while the system runs.

PART 5

Algorithms – images, OCR and addresses

5.1 Image-quality check

The photo is scaled to 1,000 px width and measured: **sharpness** = variance of the Laplacian (how strongly brightness changes at edges – blur makes it small; limit 60), **brightness** = mean grey value (60–225), **contrast** = standard deviation (≥ 25), **glare** = share of pixels ≥ 250 (≤ 6 %), and the short side ≥ 600 px.

backend/app/services/ocr/quality.py

5.2 Otsu thresholding

Chooses automatically the grey level that best separates ink from paper (it tries every level and keeps the one where the two groups are most different), producing a black-and-white image.

ocr/pipeline.py (deskew, segment_lines)

5.3 Deskew of printed text

Ink is smeared sideways (dilation with a 25 × 3 rectangle) so each line becomes one blob; the tilt of every long blob is measured with a minimum-area rectangle; the length-weighted average angle is used to rotate the image level, if it is between 0.3° and 15°.

ocr/pipeline.py · deskew()

5.4 Line segmentation (projection profile)

Count the ink pixels in every row: rows with text have many, gaps between lines have few. Each run of busy rows (at least 6 px tall) becomes a line box, trimmed left and right to the ink.

ocr/pipeline.py · segment_lines()

5.5 Ink map (flat-fielding) for handwriting

Estimate the *paper* brightness everywhere with a grey-scale **closing** (a filter that removes thin dark strokes) and a blur; then $\text{ink} = (\text{paper} - \text{pixel}) \div \text{paper}$. Shadows and uneven lamp light cancel out. The result is rescaled by the darkness of a typical stroke (90th percentile), so faint ruled lines and show-through drop out.

ocr/htr.py · ink_map()

5.6 Rule-line removal and straightening

Long horizontal rules are found with a morphological **opening** using a very wide 1-pixel-high kernel and erased. The block is then rotated to the angle (searched from −12° to +12°, first every 1°, then every 0.25°) at which the row profile is **sharpest** – the sum of squared differences between neighbouring rows is largest when text rows and gaps are cleanly separated.

ocr/htr.py · remove_rules(), straighten()

5.7 Handwritten line finding

A smoothed row profile is split into runs of ink; the typical line height is measured from the dense cores of the lines; a run that is much taller than one line (touching ascenders/descenders) is cut at its weakest row. Each line is then cropped to its writing, ignoring form lines and box edges (connected-component analysis).

ocr/htr.py · find_lines(), text_box()

5.8 CTC greedy decoding with a digit preference

For every column take the most likely character; merge repeats and drop blanks (e.g. `– – ක ක – ා – → කා`). Words are split at spaces; a word's confidence is its least-sure character. Inside a token that is clearly a number (≥ 3 digits, ≥ 60 % digits), a letter is replaced by the best digit if the network gave that digit at least 15 % (handwritten l/1, O/0), and the word stays low-confidence. Sinhala zero-width joiners are put back for conjuncts like ක්‍ර.

ocr/htr.py · decode(), restore_joiners()

5.9 Choosing the reader (P1 “auto”)

Read the block with Tesseract. If its average word confidence is ≥ 80 %, it is clean print – keep it. Otherwise read it with the handwriting model too, and keep whichever reader is **more confident**. “Handwritten” or “Printed” forces one reader.

ocr/pipeline.py · run_ocr()

5.10 Text normalisation, tokenising and CRF features

Normalisation only changes invisible things: Unicode NFC, removal of zero-width spaces and soft hyphens, straight quotes, single spaces – never letters or digits. **Tokens** keep their line number and trailing comma; “No.25” is split into marker + number. **Features per word**: the word, first/last 2–3 characters, script (Unicode block), shape (e.g. “dd/d”), digit counts, slash/hyphen/period/comma, gazetteer membership (town, district, province, multi-word names, known postcodes, Colombo), trilingual vocabulary (honorifics, house markers such as “No.”/“අංක”/“இல.”, street types such as “Road”/“පාර”/“வீதி”), position in the line and in the address, and the same clues for the two words before and after.

address/text.py · address/features.py · address/lexicon.py

5.11 Field confidence, flags and review statistics

Field confidence = the **minimum** marginal probability of its words; below the frozen threshold (0.974) the field is flagged; for handwriting, a word the reader was less than 60 % sure of lowers its field to that value. At confirmation each field’s suggestion is compared with the final value → *kept unchanged / corrected / added manually / removed*.

address/parser.py · ocr/pipeline.py (flag_uncertain_fields) · routers/intake.py (review_stats)

5.12 Temperature scaling

Networks are often over-confident. Divide the scores by one number T before the softmax: $\mathrm{p} = \mathrm{softmax}(\mathrm{scores} \div \mathrm{T})$. T is fitted on validation (L-BFGS, minimising the log-loss) and stored with the model; it does not change which answer wins, only how sure it claims to be.

ml/script_id/train.py · ocr/script_id.py

5.13 Gazetteer check, Colombo postcode and the guard

Town ↔ postcode: the town is looked up by its English, Sinhala or Tamil name in 2,155 localities; the result is consistent, mismatch, unknown_city, unknown_postcode or incomplete – it never edits a value. **Colombo zones**: “Colombo 03” → 00300, offered only as a suggestion. **Guard**: a suggested field is refused if it contains a digit not in the source text, a name/street/town word not in the source text (except approved street expansions such as Rd → Road), or a change of script.

address/lexicon.py · address/normalizer.py

PART 6

Algorithms – weighing and price

6.1 Reading the HX711 (bit-banging) and two's complement

When the data pin goes low, the ESP32 sends 25 clock pulses (4 µs high, 4 µs low) and reads one data bit after each of the first 24; the 25th pulse selects channel A, gain 128. The 24-bit number is in **two's complement**: if its top bit is 1 the value is negative, so the upper 8 bits are filled with 1s.

iot/.../postassist_scale_cam.ino · hxRead()

6.2 Sliding window, trimmed mean, stability, spikes, zero tracking

The last 10 readings (1 s) are kept. **Trimmed mean**: drop the highest and the lowest, average the rest. **Spread** = highest − lowest. **Stable** = 10 readings and spread ≤ 1 g. **Spike rejection**: a reading more than 25 g from the last accepted one is held back until the next reading agrees. **Zero tracking**: while empty and within 0.5 g of zero, the zero point moves 1/20 of the way towards the current value each reading.

postassist_scale_cam.ino · sampleScale()

6.3 Two-point linear calibration

$\text{grams} = (\text{raw} - \text{offset}) \div \text{factor}$. Tare stores *offset* = raw value when empty; calibrate stores $\text{factor} = (\text{raw} - \text{offset}) \div \text{known grams}$. Example: offset 84,300, 100 g reads 299,300 → factor 2,150 counts/g; a letter reading 127,300 → 20.0 g. Factors below 100 counts/g are refused as impossible.

postassist_scale_cam.ino · tare(), calibrate(), loadCalibration()

6.4 Placement numbering and the placement key

A small state machine: *empty* → (stable and ≥ 2 g) → *loaded*, placement + 1 → (stable and < 1 g) → *empty*. The backend builds the key $\text{device} : \text{boot ID} : \text{placement}$ and stores it under a **unique index**, so one placement can belong to one letter only.

firmware sampleScale() · services/weighing.py · db.py (indexes)

6.5 Pairing rules and the uncertainty used for pricing

A reading may be paired only if it is ≤ 3 s old, sensor OK, tared, calibrated, loaded, stable and not negative. The stored uncertainty is $\max(U, \text{spread} \div 2)$, where U is the measured expanded uncertainty or 2 g by default.

services/weighing.py

6.6 Expanded uncertainty U from the trials

Per reference mass: bias (mean error) and repeatability (standard deviation). $U = 2 \times \sqrt{(\text{largest } |\text{bias}|^2 + \text{largest } SD^2)}$ – a simple, conservative GUM-style combination of the systematic and random parts, with coverage factor k = 2 (about 95 %). Also reported: MAE, worst error, and a straight-line fit reading = a × reference + b.

ml/evaluation/weighing_analysis.py

6.7 Best photo of a burst

At a fixed JPEG quality a sharper frame has more fine detail and therefore a larger file, so of 3 frames (after 3 warm-up frames) the **largest JPEG** is kept. The focus helper uses the same idea: the frame size is the "detail score".

postassist_scale_cam.ino · captureBest(), /focus

6.8 Tariff engine

Validation: bands must start at 0 g and follow each other without gaps or overlaps ($\text{lower} < W \leq \text{upper}$), the maximum must match, a registration fee and an effective date must exist. **Selection**: exactly one tariff may be in force on the day. **Band lookup** with exact decimal numbers. **Boundary rule**: look up the bands of $W - u$ and $W + u$; if either differs from the band of W → "reweigh required". **Insurance**: $\text{ceil}(\text{value} \div 1{,}000) \times \text{Rs } 100$. Money uses Python `Decimal` rounded half-up to 2 decimals – never floating point.

services/tariff.py

PART 7

Algorithms – scans, GPS and messages

7.1 State machine and late history

A table of allowed next statuses for the forward journey and a variant for returns. A scan is valid only if its status is in the table for the current status. A scan of an *earlier* forward stage captured *before* the current status was captured is stored as history only (an offline phone uploading late).

services/state_machine.py

7.2 Role and object-level authorisation

First the role must be allowed to record that status; then the letter itself is checked: counter – own office accepted it; hub – dispatch only at origin, arrival only at destination; postman – destination office is theirs, postcode in their beat or assigned to them, attempt/delivery only if assigned; postmaster – office is origin or destination.

services/scan_events.py · authorize()

7.3 Haversine distance

Distance between two latitude/longitude points on the Earth (radius 6,371 km), accurate for short distances: $d = 2R \cdot \operatorname{asin}(\sqrt{(\sin^2(\Delta\varphi/2) + \cos\varphi_1 \cos\varphi_2 \sin^2(\Delta\lambda/2))})$.

services/location.py · haversine_m()

7.4 From GPS accuracy to a spread (σ)

Android reports accuracy as the radius that contains the true position 68 % of the time. Treating the error as a circular Gaussian, that radius is a Rayleigh quantile, so $\sigma = \text{accuracy} \div \sqrt{(-2\ln(1-0.68))}$ ≈ accuracy ÷ 1.51.

services/location.py · sigma_from_accuracy()

7.5 Probability of being inside the zone (policy L2)

If the true position is spread around the GPS point with σ, its distance from the zone centre follows a **Rice distribution**. The probability of being inside a zone of radius R is the area under that curve from 0 to R, computed numerically (trapezoid rule over 1,601 points, with an overflow-safe Bessel function). When the GPS point is exactly on the centre this becomes $1 - e\text{^}(-R^2/2\sigma^2)$. Then: ≥ 0.80 consistent, ≤ 0.05 mismatch, otherwise uncertain; older than 120 s or accuracy worse than 250 m → uncertain; no fix → unlocated.

services/location.py · p_inside_circle(), assess()

7.6 Policy action table

For each status, a table turns the location result into an action: accept, accept and flag, or hold (e.g. mismatch → hold; an unlocated *arrival* → hold). Mock location and a capture time more than 5 minutes in the future also hold the scan. Each policy has a version; switching creates a new version, and every event stores the one that decided it.

services/location.py · action_for() · routers/admin.py

7.7 Idempotency

Doing the same request twice must not create two records. **Unique indexes** on the draft ID of items, the placement key and every event ID; a second accept returns the existing item; a second upload of a scan returns “duplicate”. The phones create the event ID once (a UUID) and reuse it on every retry.

db.py · routers/intake.py · services/scan_events.py · mobile db.js / scan_queue.dart

7.8 Optimistic concurrency (revision number)

Every item has a `revision`. An update is written only “if revision is still N”, and adds 1. If someone else changed the item first, nothing is written; the code re-reads the item and re-checks (up to 5 times). No locks are needed.

services/scan_events.py · ingest_scan(), review_event()

7.9 Transactional outbox

The message is written in the **same single-document update** as the event and the status, so they exist together or not at all. **De-duplication key** = first 40 hex characters of $\text{SHA-256}(\text{event} \mid \text{recipient} \mid \text{channel} \mid \text{template version})$. A background thread wakes every 2 s, **claims** a due message with a compare-and-set (only if its status and attempt count are unchanged), sets a **60-second lease**, sends it and records the outcome. Back-off after a temporary failure: $\min(10 \times 2^{n-1}, 3{,}600)\ \text{s}$, at most 6 attempts. A lease that expires while "sending" becomes *retry* (if the provider removes duplicates) or *ambiguous*.

services/notifications/outbox.py · adapters.py

7.10 Offline queue and ordered sync

Each scan is inserted into a local SQLite table (write-ahead logging for durability) as *pending* before any upload. When online, up to 50 pending scans are sent oldest-first to `/api/scans/batch`; the server processes them in capture order and answers each one; only a lasting answer (accepted, held, rejected, duplicate) removes it from pending.

mobile_app/src/db.js, sync.js · flutter_app/lib/data/scan_queue.dart, sync_service.dart

7.11 Command queue for the device

Commands from the web app are stored as *queued*; each reading's reply hands over up to 3 of them (marked *sent*); the device reports *done/failed*; anything not picked up within 60 s expires; a photo request expires after 25 s.

routers/iot.py · routers/devices.py · routers/intake.py

PART 8

Algorithms – security and identifiers

8.1 Password hashing – PBKDF2-HMAC-SHA256

A random 16-byte **salt** is mixed with the password and hashed **390,000 times**; stored as `pbkdf2_sha256$iterations$salt$hash`. Checking repeats the work and compares with `hmac.compare_digest` (constant time, so timing reveals nothing). The many rounds make guessing very slow.

security.py

8.2 Login token – JWT (HS256)

A small signed message $\{\text{username}, \text{role}, \text{issued}, \text{expires}\}$; the signature is an HMAC-SHA256 with the server's secret, so it cannot be forged or edited. Valid for 720 minutes. On every request the user is read again from the database, so a deactivated account stops at once.

security.py · create_token(), get_current_user()

8.3 Device key, tracking token and audit

The device key is stored only as its SHA-256 hash and compared in constant time. The public tracking token is 16 random bytes (`secrets.token_urlsafe`) placed after `#` in the link; a wrong token and an unknown ID give the same 404. Every sensitive action is appended to the audit log with a severity (info, warning, security).

security.py · routers/public.py · audit.py

8.4 Tracking ID with a check character

10 characters drawn with a secure random generator from the **Crockford base-32** alphabet (0–9 and letters without I, L, O, U). The check character is the 10-character number $\bmod 37$, taken from a 37-symbol alphabet; draws whose check would be a symbol are thrown away, so IDs stay alphanumeric. Because 37 is prime, every single wrong character and every swap of neighbours changes the check. When typed, O is read as 0 and I/L as 1.

services/tracking_id.py (and flutter_app/lib/core/tracking_id.dart)

8.5 Barcode and QR

Code 128: a 1-D barcode that encodes letters, digits and "-" compactly; read by any USB scanner. **QR** with error-correction level **M**: still readable with about 15 % of it damaged. Both are drawn as SVG (vector), so they print sharp at any size; they contain only the ID (or the tracking link).

services/labels.py

8.6 Time-to-live index

MongoDB deletes raw scale readings automatically 3 days after they arrive (an index with `expireAfterSeconds`), so diagnostic data does not grow for ever.

db.py

PART 9

How the models were evaluated

Method / metric	Meaning
Train / validation / test	Learn on train, choose settings and thresholds on validation, measure once on test.
Grouped splits	Addresses split by postal town; script lines by font and text; handwriting by the published splits – so test examples are truly new.
Accuracy · precision · recall · F1	Share correct · of those predicted as X, how many are X · of the real X, how many were found · balance of the two.
Exact match	A field (or whole address) counts only if every character is right.
CER / WER	Edit distance (substitutions + deletions + insertions, Levenshtein) ÷ length of the true text, by characters / words.
ECE	Expected calibration error: group answers by confidence and compare average confidence with actual accuracy; 0 is perfect.
Multiple seeds	The script model was trained 3 times with different randomness; mean ± SD shows how stable the result is.
Monte-Carlo simulation	20,000 random scans per scenario (inside, outside, boundary, stale) with a realistic phone-error model, run through the real policy code.
Baselines	Rule parser (address), Tesseract (handwriting), B0/B1 (capture), L0/L1 (location).

PART 10

Every file – backend

`backend/`

10.1 Start-up and shared code (`app/`)

File	What it does
`main.py`	Creates the app; at start-up: starts MongoDB if needed, creates indexes, adds demo data, starts the outbox worker; attaches the 11 routers; `/api/health`; attaches the teammate's KPI desk last.
`config.py`	Reads every setting from `.env` into one `Settings` object (database, JWT secret, CORS, storage, OCR, weight limits, e-mail, demo data, MongoDB auto-start).
`db.py`	MongoDB connection; starts a local `mongod` (detached) when none answers; all indexes (unique, TTL, query indexes).

File	What it does
`security.py`	PBKDF2 hashing, JWT, `get_current_user`, `require_roles`, `get_device`.
`audit.py`	Appends one audit entry; never breaks a request if writing fails.
`schemas.py`	Pydantic request models: login, draft fields, OCR options, crop, weight, quote, accept, scan, batch, review, device command, trial, office, user …
`seed.py`	6 offices (OpenStreetMap provenance), 10 demo users, the 2026 tariff, the L2 policy, device SCALECAM-01 – only if missing.
`timeutil.py`	UTC "now" and Sri Lanka time formatting.
`reference/`	lk_localities.csv (2,155 towns), lk_post_offices_osm.csv (797), address_vocabulary.json, tariff JSON.

10.2 Routers (`app/routers/`) – 70 endpoints

File (endpoints)	What it does
`auth.py (2)`	Log in; who am I.
`intake.py (22)`	Drafts (create, list, get, update, cancel); image upload and download; scale capture and its status; OCR; parse text into a draft or alone; LLM normalise; address check; office suggestions; confirm fields; pair weight; manual weight; live weight; quote; accept; capabilities.
`items.py (8)`	List, detail, label data, label printed/reprint, label verify, assign postman, recipient pin, rotate tracking link.
`scans.py (4)`	One scan, batch upload, look-up for scanning, statuses for a role.
`review.py (4)`	Held scans, flagged scans, approve/reject, audit search.
`notifications.py (4)`	Outbox list with counts, preview, retry, resolve.
`devices.py (10)`	List devices, register, rotate key, live view, readings, send command, record/list trials, trials CSV, set uncertainty.
`iot.py (3)`	Used by the device: readings, photo upload, command result.
`public.py (1)`	Public tracking (needs the token).
`admin.py (8)`	Offices and zones, tariffs and approval, active policy and switching, users list and create.
`reports.py (4)`	intake.csv, events.csv, notifications.csv, summary.
`common.py`	Helpers: not-found, coded problems, same-office check.

10.3 Services (`app/services/`)

File	What it does
`tariff.py`	Validation, selection, band lookup, quote, boundary rule, insurance, money rounding.

File	What it does
`weighing.py`	Pairing rules, placement key, uncertainty.
`tracking_id.py`	Random IDs with check character; normalise; validate.
`labels.py`	Code 128 and QR as SVG.
`state_machine.py`	Statuses, labels, allowed moves, late history, role permissions, which statuses notify.
`scan_events.py`	Eight-check scan pipeline, reference zones, authorisation, status fields, outbox entries, review.
`location.py`	Default policy, haversine, σ from accuracy, Rice probability, assess, action table.
`item_views.py`	Staff view vs. redacted public timeline.
`images.py`	Save photo to disk, measure quality, attach to the draft.
`ocr/quality.py` · `pipeline.py` · `script_id.py` · `htr.py`	Quality gate · B1/P1 pipelines and reader choice · script-ID model · handwriting model and its pre-processing.
`address/text.py` · `lexicon.py` · `features.py` · `parser.py` · `normalizer.py`	Normalise and tokenise · gazetteer and vocabulary · CRF features · CRF / rules → fields + confidence · rule/LLM normaliser + guard.
`notifications/templates.py` · `adapters.py` · `outbox.py`	EN/SI/TA texts · file/SMTP/Resend/SMS senders · outbox entries and worker.

10.4 Models, tools, tests, settings

File	What it does
`models/address_crf.crfsuite` + `.meta.json`	The CRF and its version, threshold and test metrics.
`models/script_id.onnx` + `.meta.json`	The script CNN, its classes and temperature.
`models/htr.onnx` + `htr.meta.json`	The handwriting network, its 196-symbol character set and test results.
`models/weekly_kpi_model.joblib`	Teammate's KPI forecast model.
`tools/start_all.bat` · `start_all.ps1`	Start MongoDB, API, bridge (if set) and web app in separate windows.
`tools/device_simulator.py`	Pretends to be the scale and camera (same protocol).
`tools/device_bridge.py`	Connects a pull-only board: polls it, derives stable/placement, runs commands.
`tests/conftest.py`	Test set-up with a temporary database that is deleted afterwards.
`tests/test_units.py` · `test_workflow.py` · `test_handwriting.py` · `test_device_bridge.py` · `test_combined_api.py`	Unit tests · end-to-end T03–T17 · handwriting pre-processing · bridge · shared server with the KPI desk.
`requirements.txt` · `.env.example`	Pinned packages · settings template.

PART 11

Every file – web app

`frontend/` (registered-mail part)

File	What it does
`vite.config.js`	Port 5173, HTTPS (unless `VITE_HTTPS=false`), `@` alias, proxy of `/api` and `/health` to port 8000.
`index.html · src/main.jsx`	The page shell; starts React with the router and the login context.
`src/App.jsx`	All routes: login, public tracking, label print, the registered-mail pages inside the layout, and the KPI desk routes.
`src/api.js`	Every request goes through here: adds the token, reads JSON, turns errors into readable messages, logs out on 401; file download and protected images.
`src/auth.jsx`	Remembers the signed-in user; `can(user, …roles)`.
`src/index.css`	Tailwind set-up and navy theme; Sinhala/Tamil font fall-backs; print rules.
`components/Layout.jsx`	Grouped sidebar (Daily work, Supervision, Set-up), search bar, account menu, skip link.
`components/ui.jsx · icons.jsx`	Buttons, cards, badges, alerts, fields, tiles, page header, spinner, money and time formats; icon set.
`components/IntakeProgress.jsx`	The 5-stage stepper of the intake screen.
`components/ImagePanel.jsx`	Step 1: capture / webcam / upload, crop box, rotate, OCR options, OCR summary, typed text.
`components/AddressBlock.jsx`	Step 2: seven fields, suggestions, confidence badges, town/postcode check, derived postcode.
`components/WeightPanel.jsx`	Step 4: live weight every 0.7 s, “Use this weight”, manual weight.
`components/StatusBadge.jsx · Timeline.jsx`	Status, location, decision and message badges; event history.
`pages/Login.jsx · Dashboard.jsx`	Sign-in; overview tiles, recent activity, quick actions, intake by hour.
`pages/CounterHome.jsx · IntakeWizard.jsx`	Drafts list (resume / delete) and capability badges; the intake screen with steps 3 and 5 and the confidence tile.
`pages/LabelPrint.jsx`	Label and receipt, print/reprint, scan-back verification.
`pages/Items.jsx · ItemDetail.jsx`	Search and filter; full history, assign postman, message states.
`pages/ScanEvent.jsx`	Desk scan with browser location.
`pages/ReviewQueue.jsx · Notifications.jsx · Audit.jsx`	Approve/reject held scans; outbox; audit search.

File	What it does
`pages/Devices.jsx · Offices.jsx · Reports.jsx`	Scale & camera; zones with “use my location”; CSV exports, policy switch, tariff approval.
`pages/Track.jsx`	Public tracking page (token from the URL fragment).

PART 12

Every file – phone apps

12.1 Expo field app (`mobile_app/`)

File	What it does
`App.js · index.js · app.json`	Screen switching and session; entry point; app name, permissions.
`src/api.js`	Requests with timeouts and the token.
`src/db.js · sync.js`	SQLite queue (WAL); ordered batch upload.
`src/location.js`	One GPS fix with accuracy, age, permission and mock flag.
`src/components.js`	Buttons, colours, styles.
`src/screens/LoginScreen · HomeScreen · ScanScreen · EventScreen`	Sign-in · counts and upload · camera scanner · status, reason, acknowledgement, signature, save.

12.2 Flutter staff app (`mobile_app/flutter_app/lib/`)

File / folder	What it does
`main.dart · app.dart`	Starts the app; theme, languages, first screen.
`core/api_client.dart`	HTTP client for the backend.
`core/app_state.dart`	Server address, signed-in user, language, sign-in/out.
`core/models.dart · labels.dart · format.dart · tracking_id.dart`	Typed API objects · translated names of codes · Sri Lankan date/money/weight formats · ID check character (same rule as the backend).
`data/scan_queue.dart · sync_service.dart · location_service.dart`	Offline queue per user · ordered upload · GPS with quality.
`features/auth/ · home/ · settings/`	Sign-in · role-based home · settings and about/privacy.
`features/intake/ (drafts, wizard, controller, progress, receipt, steps/photo · address · confirm · weigh · payment)`	Counter registration in 5 steps; label and receipt with verification.

File / folder	What it does
`features/scan/ (scanner, record_scan, scan_flow, scan_history)`	Camera scanner; record a status with GPS and acknowledgement; scan-next loop; phone's scan history.
`features/items/ · review/ · summary/ · audit/ · notifications/ · offices/ · devices/ · tracking/`	Items and history · review queue · postmaster summary · audit log · sender messages · zones from GPS · scale & camera · public tracking.
`l10n/ (app_en.arb, app_si.arb, app_ta.arb + generated files)`	All screen texts in English, Sinhala and Tamil (613 strings).
`theme/gov_theme.dart · widgets/`	Colours and type; shared widgets, language switcher, signature pad, status progress, timeline.
`test/ (api_client, backend_contract, logic, tracking_id, widget)`	32 automated tests, including a contract test against a live backend.

PART 13

Every file – IoT device

`iot/postassist_scale_cam/`

File	What it does
`postassist_scale_cam.ino`	The firmware: settings and calibration in flash, HX711 read, weighing window, tare/calibrate/reset, camera set-up and best-of-burst, HTTP to the backend, commands, device web page, serial commands, Wi-Fi and set-up network, `setup()` and `loop()`.
`config.h (not in git) · config.example.h`	Wi-Fi, backend address, device ID and key, pins, weighing thresholds, camera settings · the template.
`device_page.h`	HTML + JavaScript of the device's own page (kept out of the .ino because the Arduino tool misreads JavaScript there).
`camera_pins.h`	Camera connector pins of the AI-Thinker board.
`iot/README.md`	Short build and flash notes.

PART 14

Every file – machine learning

`ml/`

File	What it does
`run_all.sh`	Rebuilds everything in order: download → reference tables → address corpus → CRF → script data → script CNN → envelopes → simulation.
`requirements.txt · README.md`	Training packages; overview of models and protocol.
`common/paths.py · fonts.py · textrender.py`	Shared paths · fonts split into train/validation/test sets · HarfBuzz + FreeType text drawing for Sinhala/Tamil.
`datasets/fetch_public_datasets.py`	Downloads every public source and records URL, licence, SHA-256 and date in MANIFEST.json.
`datasets/build_reference_data.py`	Turns downloads into clean tables (towns, post offices, streets, names) and copies run-time ones to the backend.
`address_parser/build_corpus.py`	Builds the labelled address corpus (Deepparse real + synthetic SI/TA/EN/mixed, 25 % OCR noise), grouped by town.
`address_parser/train_crf.py`	Grid search on validation, final training, threshold choice, test report.
`script_id/render_dataset.py · train.py`	Renders 54,000 degraded text lines by font split · trains 3 seeds, temperature scaling, ONNX export, report.
`htr/build_cache.py · sinhala_glyphs.py · data.py · train.py · evaluate.py`	Prepares real lines and glyphs from the archives · labels of 270 of the 454 character classes · real + synthetic data and augmentation · CRNN-CTC training and ONNX export · test vs Tesseract and report.
`synthetic_envelopes/generate.py`	360 synthetic envelope photos with exact ground truth.
`evaluation/ocr_benchmark.py · weighing_analysis.py · location_policy_simulation.py`	B1 vs P1 vs P1+HW · MAE and U from trials · L0/L1/L2 Monte-Carlo.
`reports/ (address_parser, script_id, htr, ocr_benchmark, location_policy, weighing)`	REPORT.md, metrics and logs of every run (weighing is still empty).

PART 15

Every file – datasets

`Datasets/`

File / folder	What it is
`README.md · raw/MANIFEST.json`	Every dataset, licence, citation, checksum; how to rebuild.
`raw/gazetteer_madurapa/`	Provinces, districts, cities (EN/SI/TA, postcodes) as SQL (MIT).
`raw/deepparse_worldwide/lk.parquet`	19,292 tagged Sri Lankan addresses (CC BY 4.0).
`raw/osm/post_offices.json · streets_si_ta.json`	OpenStreetMap post offices and SI/TA street names (ODbL).
`raw/wikidata/sri_lankan_person_names.csv`	Given/family names (CC0).
`raw/fonts/ · raw/slpost/`	27 Google Fonts (OFL) · the official tariff notice PDF.
`curated/address_vocabulary.json · name_seed_list.csv · tariff_….json`	Honorifics, house markers, street types, Colombo zones in EN/SI/TA · common names · the transcribed tariff.
`processed/reference/`	lk_localities, lk_post_offices_osm, lk_streets_osm, person_names, tariff.
`processed/address_tagging/ (train/val/test.jsonl, stats.json)`	The CRF corpus and its statistics.
`processed/script_id/ · synthetic_envelopes/`	Rendered lines (npz, not in git) and preview · envelope annotations (images not in git).
`templates/ (envelope_annotation, weighing_trials, location_trials, usability_task_log)`	Sheets for collecting your own evaluation data.
`%USERPROFILE%\PostAssist\datasets\`	sinocr_handwritten.zip, sinhala_chars_454.zip, iam_line/*.parquet, cache/ (outside the repository).

PART 16

Every file – documents and root files

File	What it is
README.md	Project title, component summary, links to the documents.
.gitignore · .claude/launch.json	What git ignores (.env, config.h, storage, images, model caches) · dev-server launch settings.
Documents/Requirements_Specification.md	Functional (FR01–FR18), non-functional (NFR01–NFR12), hardware, business rules, data, ML requirements, interfaces, evaluation, traceability, acceptance tests.
Documents/Setup_and_Run_Guide.md	Install, run, demo, experiments.
Documents/IoT_Hardware_Setup_Guide.md · IoT_Integration_Test_Flow.md · images/iot_wiring_diagram.svg	Hardware build and wiring · device test steps · wiring diagram.
Documents/Flutter_Mobile_App_Guide.md	Setting up and running the Flutter app.
Documents/PP1_Individual_IT23324138_verified.pptx	PP1 slides (fact-checked).
Documents/PostAssist_Folder_Structure_EN_SI.pdf	Folder guide in English and Sinhala.
Documents/PostAssist_Component_Guide_EN.pdf · _SI.pdf	The component explained simply.
Documents/PostAssist_IoT_Scale_Guide_EN.pdf	Everything about the IoT part.
Documents/PostAssist_Research_Work_Record.pdf	Everything done so far, results and what is left.
Documents/PostAssist_Technical_Reference.pdf	This document.